Lecture 7
Bash control structures

Comp 206 – Introduction to Software Systems – Fall 2026

Mathieu Bérubé-Vallières

McGill University

September 22, 2026

External announcements

Bérubé-Vallières © 2026
See last slide for complete credits

Participants needed for Social Psychology Research



- If you think you might like to participate in one of our studies and be compensated for your time (about $10/hr), you can enter our participant pool.
- **To sign up, either scan the QR code or click on the following link** to complete the 1-minute McGill Social Psychology Recruitment
- **Survey**: https://mcgill.qualtrics.com/jfe/form/SV_bmfhtNXmHyEr11Y
- Researchers in McGill's Social Psychology Labs will then have access to your information and will contact you to let you know about available studies.
- All studies are approved by the McGill Ethics Committee.
- There is no obligation to participate in a study once you are in the pool, and you are free to withdraw from the pool at any time.

Principal Investigator: Dr. John Lydon
Contact: Silvia Foute Nelong at lydon.psychology@mcgill.ca or silvia.foutenelong@mail.mcgill.ca
Note: if you signed up last semester and are interested in participating again this semester, please complete the online questionnaire again as your information may have changed.

Variables

Bérubé-Vallières © 2026
See last slide for complete credits

Last time: Variables (user-created)

- Variables are assigned using the syntax `name=value`
  - Important: no space before and after equal sign!
- To use a variable, `$name` is used
- Examples:
  - str="hello world"
  - num=5
  - date=$(date) # what does this do?
  - echo $str
  - echo $num
  - echo $date

Bérubé-Vallières © 2026
See last slide for complete credits

Not mentioned last time

- As we saw, we can access the value of variable `name` with `$name`
- An alternate way is `${name}`
- Both forms are equivalent, both in some contexts only the second form works
  - `animal="cat"`
  - `echo $animals`
    - Tries to access variable "`animals`", will output nothing (or an unexpected output)
  - `echo ${animal}s`
    - outputs `cats`

Bérubé-Vallières © 2026
See last slide for complete credits

Last time: Positional variables

- Parameters passed to a bash script (e.g. on the command line) are numbered from $1
- For example, if we run `./script 5 2 Bob`
- From the script we can access `5, 2, Bob` using $1, $2, $3

Bérubé-Vallières © 2026
See last slide for complete credits

Last time: Example

- The following script takes as input a file extension and a directory, and moves all files with that extension from the current directory to the specified directory

```
#!/bin/bash
mv *.$1 $2
```


Bérubé-Vallières © 2026
See last slide for complete credits

Last time: Example

- The following script takes as input a file extension and a directory, and moves all files with that extension from the current directory to the specified directory

```
#!/bin/bash
mv *.$1 $2
```

This script assumes the user entered the expected arguments: Will learn how to check user input today!

Bérubé-Vallières © 2026
See last slide for complete credits

Other positional variables

- System parameters:
  - $# : number of arguments on the command line
  - $- : options supplied to the shell
  - $? : exit value of the last command executed
  - $$: process number of the current process
  - $! : process number of the last command done in background
- Command-line parameters:
  - $n : where n is from 1 through 9, reading left to right
    - ${10} , ${11} …
  - $0 : the name of the current shell or program
  - $* : all arguments on the command line "$1 $2 ... $9" as a string
  - $@ : all arguments on the command line ("$1", "$2", . . ., "$9") as an array

Bérubé-Vallières © 2026
See last slide for complete credits

Example

- Consider the following script `./pos`:

```
#!/bin/bash
echo '$#:' $#
echo '$-:' $-
echo '$?:' $?
echo '$$:' $$
echo '$!:' $!
echo '$3:' $3
echo '$0:' $0
echo '$*:' $*
echo '$@:' $@
```

- What is displayed if this is executed? `./pos 3 5 7 8 9`

Bérubé-Vallières © 2026
See last slide for complete credits

Quoting

Bérubé-Vallières © 2026
See last slide for complete credits

Quotes

- Single quotes (`'`) are used to preserve the literal value of characters within the quotes
- Double quotes (") are used to preserve the literal values of characters within the quotes, except $, ` and \
  - i.e. variables, command substitution and escape characters are still processed
- In both cases, the space character is preserved. For example, compare
  - `mkdir John Smith`
    - creates two directories, “John” and “Smith”
  - `mkdir "John Smith"`
    - creates one directory, “John Smith”

Bérubé-Vallières © 2026
See last slide for complete credits

Quotes and variables

- The same applies to variables
  - `x="John Smith"`
  - `mkdir $x`
    - shell expands this to `mkdir John Smith`
    - creates two directories, “John” and “Smith”
  - `mkdir "$x"`
    - shell expands this to `mkdir "John Smith"`
    - creates one directory, “John Smith”

Bérubé-Vallières © 2026
See last slide for complete credits

Escape characters

- The backslash (\) is Bash's escape character
- It preserves the literal value of the next character that follows
  - For example: \" is the literal single quote character, as opposed to the start of a quote
- With some commands and some characters, it can instead toggle on a special meaning for the character:
- For example, when using `echo -e`
  - `\n` means newline
  - `\r` means return
  - `\t` means tab

Bérubé-Vallières © 2026
See last slide for complete credits

Control flow

Bérubé-Vallières © 2026
See last slide for complete credits

Exit status

- Any program, when it exits, gives a *status code* back to the OS to signal whether the command succeeded or failed
  - Also called *exit status*
- Code 0 means success, and any other code means failure
  - Commonly, code 1 is used for failure
- You an end a script by using the `exit` command, optionally specifying a status code, e.g. `exit 1`
  - The default code is 0.

Bérubé-Vallières © 2026
See last slide for complete credits

Exit status

- Any program, when it exits, gives a *status code* back to the OS to signal whether the command succeeded or failed
  - Also called *exit status*
- Code 0 means success, and any other code means failure
  - Commonly, code 1 is used for failure
- You an end a script by using the `exit` command, optionally specifying a status code, e.g. `exit 1`
  - The default code is 0.
- Each time we run a command, either interactively in the terminal, or via a script, the shell updates the variable $? with the exit status of the last command we ran.
  - Can check exit status of previous command with `echo $?`
    - Note: After you do this, the exit status will be replaced by that of the echo command!

Bérubé-Vallières © 2026
See last slide for complete credits

Running commands one after another

- There are multiple ways to chain commands together
  - One way that we saw is redirection/piping
  - Another way that we saw is to write a script, where each line is executed one after the other
- Another way is the semicolon (;), with the same meaning as a new line in a script:
  - `mkdir my_dir ; cd my_dir # runs mkdir my_dir, then cd my_dir`

Bérubé-Vallières © 2026
See last slide for complete credits

Running commands one after another

- There are multiple ways to chain commands together
  - One way that we saw is redirection/piping
  - Another way that we saw is to write a script, where each line is executed one after the other
- Another way is the semicolon (;), with the same meaning as a new line in a script:
  - `mkdir my_dir ; cd my_dir # runs mkdir my_dir, then cd my_dir`
- A variant of this that we'll discuss later is the double ampersand (&&), which executes the second command only if the first one exits successfully
  - `mkdir my_dir && cd my_dir # runs mkdir my_dir, then if mkdir exited with 0, runs cd my_dir`

Bérubé-Vallières © 2026
See last slide for complete credits

If statement

```
if COMMAND
then
    ...
elif COMMAND
then
    ...
else
    ...
fi
```


Bérubé-Vallières © 2026
See last slide for complete credits

If statement

```
if COMMAND
then
    ...
elif COMMAND
then
    ...
else
    ...
fi
```

optional

Bérubé-Vallières © 2026
See last slide for complete credits

If statement

```
if COMMAND
then
    ...
elif COMMAND
then
    ...
else
    ...
fi
```

If-statement runs a given command, and if it succeeds (i.e. exits with code 0), it runs the block of code in the `then` branch

Bérubé-Vallières © 2026
See last slide for complete credits

If statement

```
if COMMAND; then
    ...
elif COMMAND; then
    ...
else
    ...
fi
```

Will often see this written with semicolon instead (remember that the semicolon has the same meaning as a new line in a script!)

Bérubé-Vallières © 2026
See last slide for complete credits

While loop

```
while COMMAND
do
    ...
done
```

Works the same way as if-statement:

While COMMAND exits with code 0, the body of the while-loop is executed

Bérubé-Vallières © 2026
See last slide for complete credits

While loop

```
while COMMAND; do
    ...
done
```

Works the same way as if-statement:

While COMMAND exits with code 0, the body of the while-loop is executed

Bérubé-Vallières © 2026
See last slide for complete credits

Comparison (`test`)

- Often, we want a condition in an if-statement or while loop to check if two variables are equal, or if a variable is equal to a particular string, etc.
- The command for this is called `test`

Bérubé-Vallières © 2026
See last slide for complete credits

Comparison (`test`): example

```
if test "$1" = "hello" ; then
    echo "Hello World!"
fi
```


Bérubé-Vallières © 2026
See last slide for complete credits

Comparison (`test`): example

```
if test "$1" = "hello" ; then
    echo "Hello World!"
fi
```

- What arguments does test receive?

Bérubé-Vallières © 2026
See last slide for complete credits

Comparison (`test`): example

```
if test "$1" = "hello" ; then
    echo "Hello World!"
fi
```

- What arguments does test receive?

1. The first string to consider
2. The equals sign, to indicate a string comparison
3. The second string to consider

Bérubé-Vallières © 2026
See last slide for complete credits

Comparison (`test`)

- To make the use of `test` look more like traditional programming, it has an alias called `[`
  - i.e. there is literally a program called `[`, you can try `man [`
- `[` works exactly like `test`, except that it expects its last argument to be `]`

Bérubé-Vallières © 2026
See last slide for complete credits

Comparison ([): example

```
if [ "$1" = "hello" ] ; then
    echo "Hello World!"
fi
```


Bérubé-Vallières © 2026
See last slide for complete credits

Comparison ([): example

```
if [ "$1" = "hello" ] ; then
    echo "Hello World!"
fi
```

- What arguments does `[` receive?

Bérubé-Vallières © 2026
See last slide for complete credits

Comparison ([): example

```
if [ "$1" = "hello" ] ; then
    echo "Hello World!"
fi
```

- What arguments does `[` receive?

1. The first string to consider
2. The equals sign, to indicate a string comparison
3. The second string to consider
4. `]`

Spacing is important!

Bérubé-Vallières © 2026
See last slide for complete credits

`test` flags (files)

Use `man test` to learn about the different comparisons you can use

Here are some useful ones to use with files:

- `-f FILE`: if FILE exists and is a regular file
- `-d FILE`: if FILE exists and is a directory
- `-x FILE`: if FILE exists and is executable

Bérubé-Vallières © 2026
See last slide for complete credits

`test` flags (integers)

Use `man test` to learn about the different comparisons you can use

Here are some useful ones to compare integers:

- `N1 -eq N2`: if N1 equals N2
- `N1 -ne N2`: if N1 is not equal to N2
- `N1 -lt N2`: if N1 is less than N2
- `N1 -le N2`: if N1 is less than or equal to N2
- (similar for `gt` and `ge`)

Bérubé-Vallières © 2026
See last slide for complete credits

`test` flags (strings)

Use `man test` to learn about the different comparisons you can use

Here are some useful ones to test strings:

- `-n STRING`: if string length is nonzero
- `-z STRING`: if string length is zero
- `S1 = S2`: if strings S1 and S2 are equal
- `S1 != S2`: if strings S1 and S2 are not equal

Bérubé-Vallières © 2026
See last slide for complete credits

`test` not

Use `man test` to learn about the different comparisons you can use

Can use ! in front of an expression to invert the outcome (condition evaluates to true if expression is false)

Example:

```
if [ ! –f $my_file]; then
        echo “$my_file doesn’t exist!”
fi
```


Bérubé-Vallières © 2026
See last slide for complete credits

Careful!

- The following two are essentially equivalent:

```
if COMMAND ; then
    ...
fi
```

```
COMMAND
if [ $? = 0 ] ; then
    ...
fi
```

- However, second way is more error-prone!

Bérubé-Vallières © 2026
See last slide for complete credits

Careful!

- However, second way is more error-prone!
- Common mistake:

```
COMMAND
echo "The exit status was $?"
if [ $? = 0 ] ; then
    ...
else
    ...
fi
```

- `echo` updates $? to 0, so else branch will never be reached

Bérubé-Vallières © 2026
See last slide for complete credits

`read`: getting input from the user

- When we run a script from the terminal, it inherits all the file descriptors of the shell that runs the script.
  - Therefore, the script's stdin is connected to the terminal.
- Bash provides a built-in command called `read` that reads from stdin into a variable.
  - Reminder that you can find info about the built-in commands by typing `man builtins`
- Example:

```
echo "How old are you?"
read -r age
echo "You are $age years old"
```


Bérubé-Vallières © 2026
See last slide for complete credits

`read`: getting input from the user

- When we run a script from the terminal, it inherits all the file descriptors of the shell that runs the script.
  - Therefore, the script's stdin is connected to the terminal.
- Bash provides a built-in command called `read` that reads from stdin into a variable.
  - Reminder that you can find info about the built-in commands by typing `man builtins`
- Example:

```
echo "How old are you?"
read -r age
echo "You are $age years old"
```

exercise: use `man builtins` to find what –r does

Bérubé-Vallières © 2026
See last slide for complete credits

`read` as a condition

- The `read` command gives exit status zero when stdin is still open and when it successfully reads a line
  - Therefore we can use read as the COMMAND in a while loop to loop over lines of stdin.
- Example:

```
while read -r line ; do
    echo "Here's a line: $line"
done
```


Bérubé-Vallières © 2026
See last slide for complete credits

Example

- By also using a pipe, we can feed the output of a command into such a while loop, enabling us to loop over the lines of output produced by a program.

```
ls | while read -r file ; do
    if [ -d "$file" ] ; then
        echo "Is a directory: $file"
    else
        echo "Is not a directory: $file"
    fi
done
```


Bérubé-Vallières © 2026
See last slide for complete credits

Example

- This program asks the user if they want to proceed before either doing the thing, or exiting.

```
read -p "Are you sure? [y/N] " -r answer
if [ "$answer" != "y" ] ; then
    exit 0
fi
```


Bérubé-Vallières © 2026
See last slide for complete credits

`shift` example

- This script prints each command-line argument on its own line.

```
while [ $# -gt 0 ] ; do
    echo "$1"
    shift
done
```

- shift is a builtin command that "shifts" over all the numbered variables. That is, the value of $2 moves into $1, the value of $3 moves into $2, and so on. The value of $1 is lost.
- By using shift, we reduce the count of command-line argument by one, which is why $# eventually reaches zero, ending the loop.

Bérubé-Vallières © 2026
See last slide for complete credits

For loop

```
for NAME in LIST
do
    COMMANDS
done
```


Bérubé-Vallières © 2026
See last slide for complete credits

For loop

```
for NAME in VALUE1 VALUE2 VALUE3
do
    COMMANDS
done
```

$NAME is set to VALUE1 on first iteration, VALUE2 on second iteration, etc.

Bérubé-Vallières © 2026
See last slide for complete credits

For loop example

```
for f in * ; do
    echo "$f"
done
```

What does this program do?

Bérubé-Vallières © 2026
See last slide for complete credits

Next class

- Bash case statements
- Bash functions
- More commands

Bérubé-Vallières © 2026
See last slide for complete credits

Additional credits

- Some content from this lecture is based on:
  - Past Comp 206 slides/notes from Prof. Vybihal and Prof. Errington

Bérubé-Vallières © 2026
See last slide for complete credits